
How to Configure JDSSArrow

The GatewayConfig model, config files, env vars and runtime edits

JDSSArrow - a SOLID, pluggable reference implementation of the Joint Dismounted Soldier System (NATO STANAG 4677, specified in AEP-76, 5 volumes), with a web configuration and monitoring application.

Reference / educational software implementing a published NATO interoperability standard. Not accredited or production tactical software.

1. The configuration model

A single GatewayConfig object fully describes a node: its identity, which plugin implements each extension point, and the network parameters. Because every pluggable choice is just a string naming a registry entry (codec: "xml", transport: "udp"), reconfiguring the system - even swapping in a third-party plugin - never requires a code change.

Layering (lowest to highest precedence)

- Field defaults - a bare GatewayConfig() is already runnable.
- Config file - a YAML or TOML file overlaid on the defaults.
- Environment variables - JDSS_-prefixed vars override the file.
- Runtime overrides - edits pushed from the web UI (PUT /api/config).

The config file is chosen by the JDSS_CONFIG environment variable (or the --config flag on the CLI). With no file, the built-in defaults are used - so the app starts with zero configuration.

2. A fully annotated config file

Every section below is optional; omit a section to accept its defaults. This example shows all of them (YAML; TOML is also supported).

```

# node-a.yaml - a coalition node (nation ALFA), also the policy authority
identity:
  node_id: node-a          # unique id on the network
  callsign: ALFA-1
  unit: 1PL/A/1-501
  nation: BEL
  role: team_leader        # rifleman | team_leader | medic | scout | ...

plugins:                   # each value names a registered plugin
  codec: xml                # Vol II - xml | json | arrow
  transport: udp           # Vol IV - udp | loopback
  security: psk            # Vol I - psk (HMAC-SHA256) | null
  bearer: simulated        # Vol III
  allocator: default       # Vol V - derives the multicast group from network_id

network:
  network_id: coalition-alpha
  multicast_group:         # optional; blank => derived by the allocator
  multicast_port:         # optional; blank => derived
  repeat: 3               # send each frame N times (1-8) for reliability
  psk: shared-coalition-secret # coalition pre-shared key - must match every node

classification:
  level: 1                # 0=UNCLASSIFIED 1=RESTRICTED 2=CONFIDENTIAL 3=SECRET
  releasable_to: REL BEL NLD

connections:
  policy: matrix           # matrix (manageable) | allow_all
  default_action: allow    # allow | deny
  blocked: []             # per-peer local blocks
  allowed: []
  policy_authority: node-a # this node owns the coalition-wide policy
  coalition_default_action: allow
  coalition_blocked: []
  authority_private_key: <hex> # ONLY on the authority - signs updates (Ed25519)
  authority_public_key: <hex> # on EVERY node - verifies updates

capabilities:              # per-message-type permissions (empty => all allowed)
  receive: {}              # e.g. { Sketch: false } drops inbound Sketch
  emit: {}                 # e.g. { CasevacRequest: false } forbids originating it

gossip:
  enabled: true            # broadcast this node's matrix row for the live matrix
  interval_s: 2.0

web:
  host: 0.0.0.0
  port: 8000
  telemetry_capacity: 1000
  peer_timeout_s: 15      # a peer is 'connected' if seen within this many seconds

servers:
  # CoT/TAK servers to bridge to (CRUD from the UI)
  - id: ots                # stable id (auto-generated when added from the UI)
    name: OpenTAKServer
    host: 192.168.0.202
    port: 8088             # 8088 plaintext, or 8089 with tls: true

```

```
tls: false
verify: true          # verify server cert (TLS only)
client_cert:          # PEM path for mutual TLS (CN = a registered OTS user)
client_key:
cacert:
enabled: true         # enabled entries connect live and bridge JDSS <-> CoT
```

3. Section reference

Section	Key fields (defaults)	Notes
identity	node_id, callsign, unit, nation, role	Who this node is; stamped on every originated message.
plugins	codec=xml, transport=udp, security=psk, bearer=simulated, allocator=default	Each value must name a registered plugin.
network	network_id=default, repeat=2, psk=..., multicast_group/port=derived	PSK and network_id must match across the coalition.
classification	level=0, releasable_to=ALL	Default marking; level 0-3.
connections	policy=matrix, default_action=allow, policy_authority=None	Admit/deny matrix and coalition policy authority + Ed25519 keys.
capabilities	receive={}, emit={}	Per-type on/off; empty means fully permissive.
gossip	enabled=true, interval_s=2.0	Peer-digest gossip feeding the live connection matrix.
web	host=0.0.0.0, port=8000, peer_timeout_s=15	Backend bind + monitoring parameters.
servers	list of {id, name, host, port, tls, verify, certs, enabled}	CoT/TAK servers to bridge to; CRUD from the Configuration tab, applied live.

4. Environment variable overrides

Every field can be overridden with a JDSS_-prefixed environment variable. Nested fields use a double-underscore delimiter (env_nested_delimiter). This is the most convenient way to tweak a container without editing a file.

```
# override nested fields: JDSS_<SECTION>__<FIELD>
export JDSS_IDENTITY__CALLSIGN=BRAVO-2
export JDSS_NETWORK__NETWORK_ID=coalition-bravo
export JDSS_NETWORK__PSK=another-shared-secret
export JDSS_CLASSIFICATION__LEVEL=2

# point the app at a config file (CLI flag or env var)
export JDSS_CONFIG=/config/node.yaml
uvicorn jdssarrow.web.app:app
```

5. Runtime configuration from the web UI

The Configuration tab edits the plugin selection, reliability (repeat) and classification live. Applying issues PUT /api/config, which hot-restarts the gateway on the new selection. If the app was started with a config file, the edit is also persisted back to that file (via the FileConfigStore); otherwise it is applied in memory only.

The capability matrix (Configuration tab) toggles per-message-type receive/emit permissions live via GET/POST /api/capabilities. A disallowed inbound type is dropped with reason 'capability'; emitting a disallowed type raises CapabilityError (HTTP 403).

6. Coalition authority signing keys

For a coalition-wide policy authority you need an Ed25519 keypair. Generate one and place the private key ONLY on the authority node; every node (including the authority) carries the public key.

```
jdssarrow keygen
# prints:
#   authority_private_key: <hex>   # -> only the authority's config, under connections:
#   authority_public_key:  <hex>   # -> every node's config, under connections:
```

7. Adding your own plugin

Because plugins are resolved from `importlib.metadata` entry points, a third party ships a new codec/transport/security/policy/profile by registering an entry point in the matching group - no fork required. For example, in another package's `pyproject.toml`:

```
[project.entry-points."jdssarrow.codecs"]
cbor = "my_pkg.cbor_codec:CborCodec"

# then simply select it:  plugins.codec: cbor
```

8. Configuring the Docker deployment

The container runs with the built-in defaults unless you provide a config. The deploy script mounts a file read-only at /config/node.yaml and sets JDSS_CONFIG for you; you can also inject individual JDSS_ env vars.

```
# mount a config file when deploying
CONFIG=examples/node-a.yaml ./deploy.sh

# change the published port
PORT=9000 ./deploy.sh

# or with docker directly
docker run -d -p 8000:8000 \
  -v $PWD/examples/node-a.yaml:/config/node.yaml:ro \
  -e JDSS_CONFIG=/config/node.yaml \
  jdssarrow:latest
```